Untitled

TP1 — Architecture SOA et services web

Domaine choisi : Gestion d'un magasin de produits cosmétiques

1. Création du document XML

J'ai choisi le domaine de la gestion d'un magasin de produits cosmétiques. Dans le fichier produits.xml, chaque produit contient un identifiant, un nom, un prix et une indication précisant s'il est en promotion. L'élément magasin est l'élément racine qui contient les différents produits.

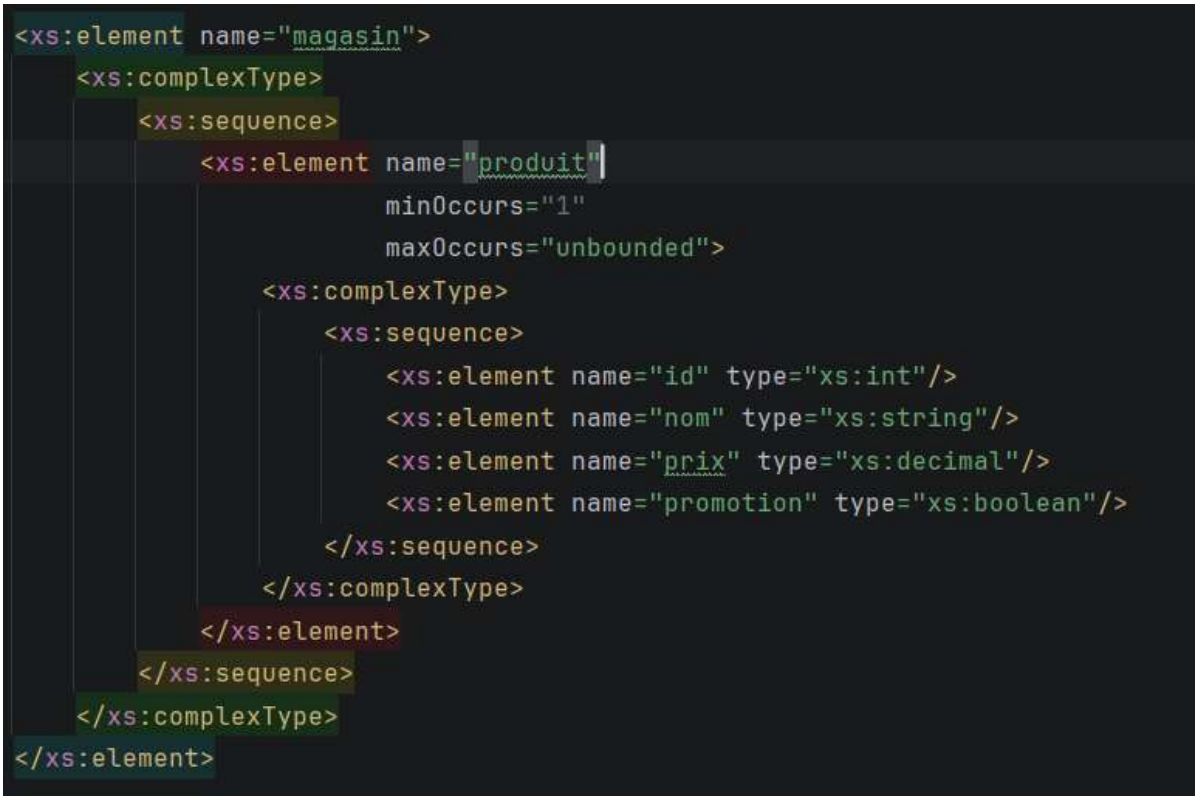
```xml
<?xml version="1.0" encoding="UTF-8"?>
<magasin>
    <produit>
        <id>1</id>
        <nom>Creme hydratante</nom>
        <prix>25.5</prix>
        <promotion>true</promotion>
    </produit>

    <produit>
        <id>2</id>
        <nom>Shampooing</nom>
        <prix>18.0</prix>
        <promotion>false</promotion>
    </produit>

    <produit>
        <id>3</id>
        <nom>Parfum</nom>
        <prix>80.0</prix>
        <promotion>true</promotion>
    </produit>
</magasin>
```

2. Définition du schéma XSD

J'ai créé le fichier produits.xsd pour définir la structure attendue du document XML. Il précise les éléments obligatoires d'un produit ainsi que leurs types : identifiant entier, nom texte, prix décimal et promotion booléenne. Il impose également l'ordre des éléments dans chaque produit.



```xml
<xs:element name="magasin">
    <xs:complexType>
        <xs:sequence>
            <xs:element name="produit"
                        minOccurs="1"
                        maxOccurs="unbounded">
                <xs:complexType>
                    <xs:sequence>
                        <xs:element name="id" type="xs:int"/>
                        <xs:element name="nom" type="xs:string"/>
                        <xs:element name="prix" type="xs:decimal"/>
                        <xs:element name="promotion" type="xs:boolean"/>
                    </xs:sequence>
                </xs:complexType>
            </xs:element>
        </xs:sequence>
    </xs:complexType>
</xs:element>
```

3. Implémentation de la validation XML/XSD

J'ai utilisé Java et les classes SchemaFactory, Schema et Validator pour vérifier que le fichier produits.xml respecte la structure définie dans produits.xsd. Si le document ne respecte pas le schéma, une erreur de validation est signalée.

```java
try {
    // 1. Validation du document XML avec le schema XSD
    SchemaFactory schemaFactory =
            SchemaFactory.newInstance(
                    XMLConstants.W3C_XML_SCHEMA_NS_URI);

    InputStream xsdStream =
            Main.class.getResourceAsStream( name: "/produits.xsd");

    InputStream xmlStream =
            Main.class.getResourceAsStream( name: "/produits.xml");

    if (xsdStream == null || xmlStream == null) {
        throw new Exception(
                "Fichier XML ou XSD introuvable !");
    }

    Schema schema =
            schemaFactory.newSchema(
                    new StreamSource(xsdStream));

    javax.xml.validation.Validator validator =
            schema.newValidator();

    validator.validate(new StreamSource(xmlStream));

    System.out.println("Validation XML/XSD reussie !");
```

4. Interrogation XPath

J'ai utilisé XPath pour extraire des informations du document XML. La première requête récupère les noms de tous les produits. La deuxième sélectionne uniquement les produits dont la valeur de promotion est true.

```java
// Requete XPath : afficher tous les noms
NodeList noms = (NodeList) xpath.evaluate(
        expression: "/magasin/produit/nom",
        document,
        XPathConstants.NODESET);

System.out.println("\nTous les produits :");

for (int i = 0; i < noms.getLength(); i++) {
    System.out.println("- " +
            noms.item(i).getTextContent());
}
```

5. Test et vérification

Après l'exécution de la méthode main() dans IntelliJ IDEA, le programme valide le document XML puis affiche les résultats des requêtes XPath. Le résultat attendu est le suivant :

```
Validation XML/XSD reussie !

Tous les produits :
- Creme hydratante
- Shampooing
- Parfum

Produits en promotion :
- Creme hydratante
- Parfum
```

Conclusion

Ce TP m'a permis de représenter des données en XML, de définir un schéma XSD pour contrôler leur structure et d'utiliser XPath pour rechercher des informations dans un document XML depuis une application Java avec Maven